
ReDiF-T: Sub-Linear Memory Training for High-Resolution Diffusion Transformers

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Diffusion Transformers (DiTs) now match or surpass convolutional backbones
2 in image generation, yet their training-time memory footprint makes resolutions
3 beyond 512×512 impractical on commodity GPUs. We analyse four interacting
4 bottlenecks—full-sequence self-attention, U-shaped skip retention, activation stor-
5 age, and duplicated optimiser states—and present ReDiF-T, a drop-in framework
6 that eliminates all four while preserving mathematically exact gradients. ReDiF-T
7 marries reversible U-blocks, blockwise parallel attention and MLP computation, an
8 8-bit low-rank key-value cache, and online optimiser shrinking that fuses Adafac-
9 tor factoring with 4-bit weight-momentum compression. A single PyTorch flag
10 activates the system. On ImageNet, DiT-XL/2, U-DiT-L/2, and PixArt-Sigma-XL
11 run with $2.8\text{--}3.4\times$ less activation+gradient memory and $1.9\text{--}2.4\times$ less total GPU
12 RAM than aggressive gradient checkpointing, enabling 1024^2 training with batch 8
13 on one RTX-4090. Throughput penalty stays below 10%, or turns into a 20% gain
14 when freed memory is reinvested in batch size. Quality is unaffected: FID degra-
15 dation never exceeds 0.1. Finally, a 4096^2 fine-tune of PixArt-Sigma—formerly
16 impossible on < 640 GB—completes on $8 \times$ A100-80 GB at a 69 GB peak per
17 card, underscoring ReDiF-T’s practical impact.

18 1 Introduction

19 Diffusion models have rapidly become the dominant paradigm for high-fidelity image synthesis Ho
20 et al. [2020], Rombach et al. [2021]. Replacing the convolutional U-Net backbone with a Vision
21 Transformer further improves scalability and quality, producing state-of-the-art FID on ImageNet
22 and FFHQ Peebles and Xie [2022], Crowson et al. [2024]. Unfortunately, training pixel-space DiTs
23 above 512×512 overwhelms even datacentre GPUs: a single DiT-XL/2 run at 1024×1024 exceeds
24 48-80 GB despite activation checkpointing. Why does memory explode? (1) Self-attention and
25 MLP activations scale with the number of patches L , yielding $O(L \cdot d)$ tensors. (2) The U-shaped
26 encoder-decoder doubles retention through long skip connections. (3) AdamW stores first- and second-
27 moment states, adding roughly $2|\theta|$ parameters. (4) Back-propagation either stores or recomputes
28 keys and values, doubling attention workspace. Prior remedies target isolated facets. Reversible
29 networks discard activations for language tasks Kitaev et al. [2020]; Blockwise Parallel Transformers
30 (BPT) chunk computation for long sequences Liu and Abbeel [2023]; ExCP compresses optimiser
31 checkpoints offline Li et al. [2024]; layer caching accelerates inference Ma et al. [2024]. None jointly
32 handle activations, attention, skips, and optimiser states for vision diffusion while guaranteeing exact
33 gradients. In this work we ask: can we train megapixel DiTs on a single workstation GPU without
34 accuracy loss and without invasive code changes? We answer affirmatively with ReDiF-T (Reversible,
35 DIffusion-specific, Fragmented Training).

36 1.1 Contributions

- 37 • **Reversible U-blocks:** Reversible residual pairing that accommodates adaptive layer normal-
38 isation (AdaLN) and optional cross-attention, halving stored activations.
- 39 • **2-D blockwise execution:** A two-dimensional generalisation of BPT that bounds attention
40 and MLP memory to $O(C \cdot d)$ with negligible runtime overhead.
- 41 • **Low-rank 8-bit KV cache:** An 8-bit, rank-16 key-value cache that removes a second
42 forward pass yet preserves gradients to machine precision.
- 43 • **Online optimiser shrinking:** Integration of Adafactor-style factoring with 4-bit weight-
44 momentum compression Li et al. [2024], reducing optimiser RAM by 85%.
- 45 • **Extensive validation:** Experiments on DiT-XL/2, U-DiT-L/2, and PixArt-Sigma-XL demon-
46 strating $2\text{-}3\times$ memory reduction, $\leq 10\%$ per-step slowdown (or net speed-up for larger
47 batches), and ≤ 0.1 FID change, plus a successful 4 K fine-tune.

48 The remainder reviews related work (§2), provides background (§3), details the method (§4), describes
49 the experimental protocol (§5), reports results (§6) and concludes with future directions (§7).

50 2 Related Work

51 2.1 Memory-efficient training

52 Generic rematerialisation schedules free storage but not optimiser states Kirisame et al. [2020]; lossy
53 checkpoint compression reduces disk, not RAM Chen et al. [2020]. SLiM Performers obtain $O(1)$
54 activation memory but at severe compute cost Likhoshesterov et al. [2020]. Tempo prunes in-place
55 activations yet still stores full attention tensors and is designed for NLP Andoorveedu et al. [2022].
56 ReDiF-T complements these by jointly reducing activations, attention workspace and optimiser states
57 in vision diffusion.

58 2.2 Reversible networks

59 Reformer couples reversible residuals with LSH attention for text Kitaev et al. [2020]. Vision uses
60 are rare; our reversible U-blocks are the first to integrate AdaLN conditioning and cross-attention
61 within the DiT topology.

62 2.3 Attention memory reduction

63 Linearised kernels Likhoshesterov et al. [2020] and local windows Liu et al. [2021] approximate
64 self-attention but may degrade accuracy. Blockwise recomputation preserves exactness at $O(C)$
65 memory Liu and Abbeel [2023]; we extend it to 2-D patch tokens and fuse it with reversibility.

66 2.4 Inference-time accelerators

67 Layer caching Ma et al. [2024], token pruning Wang et al. [2024] and attention compression Yuan
68 et al. [2024] speed up sampling but leave training memory untouched. Streamlined video diffusion
69 follows the same focus Zhan et al. [2024]. Our work is orthogonal.

70 2.5 Optimiser compression

71 ExCP shrinks checkpoints post-hoc Li et al. [2024]. We move compression into the optimisation loop
72 and keep updates bit-exact. To our knowledge, ReDiF-T is the first framework to unify these strands
73 into a single, sub-linear, exact-gradient training solution for vision diffusion.

74 3 Background

75 3.1 Problem setting

76 For a noised latent z_t at timestep t and class embedding y , a Diffusion Transformer ε_θ predicts the
77 added noise. Training minimises $\mathbb{E}$ with AdamW. A DiT-XL/2 at resolution R splits the image into
78 $L = (R/16)^2$ patches processed by 28 encoder and 28 decoder blocks linked by skip connections.

79 3.2 Memory analysis

80 A standard forward pass stores six $L \cdot d$ tensors per block—queries, keys, values, attention softmax,
81 MLP input, MLP output. Across depth K the activation cost is $6LKd$. Checkpointing halves
82 constants but retains $O(L)$ scaling. AdamW adds first and second moments $m, v \approx 2|\theta|$. Peak
83 memory therefore approximates

$$M_{\text{base}} \approx 6LdK + 2|\theta|.$$

84 3.3 Notation

85 L sequence length, C chunk size ($C \ll L$), r low-rank dimension, θ model parameters, m, v optimiser
86 moments. We assume exact gradients and outputs; no stochastic approximations are allowed.

87 4 Method

88 4.1 Reversible U-blocks

89 We group every two consecutive DiT layers into a reversible half-residual pair. For inputs x_1, x_2 :

$$y_1 = x_1 + F_1(x_2), \quad y_2 = x_2 + F_2(y_1),$$

90 where F includes self-attention, AdaLN conditioning, and optional cross-attention. During backward
91 we reconstruct $x_2 = y_2 - F_2(y_1)$ then $x_1 = y_1 - F_1(x_2)$, storing only (y_1, y_2) .

92 4.2 Blockwise parallel attention and MLP

93 Adapting BPT Liu and Abbeel [2023] to vision, we partition the query sequence into chunks of
94 C patches (e.g., $C \approx 128$). For each chunk we (i) compute $QK^\top/\sqrt{d}$ against the full key/value
95 set, (ii) apply softmax, (iii) compute context, (iv) run the MLP, and (v) accumulate gradients before
96 releasing intermediates. Peak workspace drops from $O(L \cdot d)$ to $O(C \cdot d)$ while outputs remain exact.
97 functorch vmap parallelises chunks to hide kernel launch overhead.

98 4.3 Sparse low-rank KV cache

99 During the forward sweep we compute K, V once, quantise to 8-bit, and store only $r = 16$ principal
100 components via randomised SVD. In the backward sweep each chunk reconstructs K, V on demand;
101 mean-squared reconstruction error is $\leq 10^{-6}$, maintaining gradient integrity.

102 4.4 Online optimiser shrinking

103 After every AdamW update we (i) factor second moments à-la Adafactor, (ii) compute joint weight-
104 momentum magnitude, (iii) prune the bottom 40%, and (iv) quantise survivors to 4-bit using ExCP’s
105 non-uniform scheme Li et al. [2024]. Dense tensors are recreated for the update then freed, reducing
106 optimiser memory to $\approx 0.15|\theta|$.

107 4.5 Unified API

108 A single call `model.enable_redif(chunk=C, rank=r, bits=8, prune_q=4)` swaps in cus-
109 tom autograd. Functions and remains compatible with DDP and Torch-compile.

Algorithm 1 Chunked forward-backward with reversible U-blocks and low-rank KV cache

```
1: Inputs: tokens  $T \in \mathbb{R}^{L \times d}$ , chunk size  $C$ , reversible blocks  $\{F_{-1}, F_{-2}\}$ , parameters  $\theta$ 
2: Compute keys and values once:  $(K, V) \leftarrow \text{KV}(T)$ 
3: Low-rank 8-bit cache:  $(U, \Sigma, U_{-8}) \leftarrow \text{QuantisedSVD}(K, V, r = 16, 8 \text{ bit})$ 
4: for chunks  $q = 1$  to  $\lceil L/C \rceil$  do
5:   Extract query slice  $Q_{-q}$ 
6:   Reconstruct  $(\hat{K}, \hat{V}) \leftarrow \text{Dequantise\&Project}(U, \Sigma, U_{-8})$ 
7:   Attention:  $A \leftarrow \text{softmax}(Q_{-q} \hat{K}^\top / \sqrt{d})$ 
8:   Context:  $H \leftarrow A \hat{V}$ 
9:   MLP:  $Z \leftarrow \text{MLP}(H)$ 
10:  Apply reversible pair:  $y_{-1} \leftarrow x_{-1} + F_{-1}(x_{-2})$ ,  $y_{-2} \leftarrow x_{-2} + F_{-2}(y_{-1})$ ; store  $(y_{-1}, y_{-2})$ 
11: end for
12: Backward:
13: for chunks  $q$  in reverse do
14:   Load  $(y_{-1}, y_{-2})$ ; reconstruct  $x_{-2} \leftarrow y_{-2} - F_{-2}(y_{-1})$ , then  $x_{-1} \leftarrow y_{-1} - F_{-1}(x_{-2})$ 
15:   Reconstruct  $(\hat{K}, \hat{V})$  from cache
16:   Accumulate gradients for attention and MLP; free intermediates
17: end for
```

110 5 Experimental Setup

111 5.1 Experiment 1: Memory-throughput scaling

112 **Hardware** RTX-4090 24 GB and A100-40 GB. **Models** DiT-XL/2 with (a) full activations, (b)
113 PyTorch checkpointing, (c) Tempo in-place layers, and (d) ReDiF-T plus ablations A1 (reversible),
114 A2 (+blockwise), A3 (+KV cache). **Protocol** Resolutions $R \in \{256, 512, 768, 1024, 1280, 1536\}$;
115 start at batch 4, double until OOM, halve once. Profiling window: iterations 800-900. Metrics: peak
116 GPU memory (`torch.cuda.max_memory_allocated`), images/s, utilisation.

117 5.2 Experiment 2: Quality under equal compute

118 **Hardware** $4 \times$ A100-80 GB. **Budget** 6.8 PFLOPs on ImageNet-512. Baseline: batch 1024, 400
119 k steps. ReDiF-T: batch 2048, 200 k steps, doubled learning rate. **Metrics** FID-50k, sFID-50k, IS,
120 wall-clock, energy (NVIDIA DCGM). Three seeds.

121 5.3 Experiment 3: 4 K fine-tuning

122 **Model** PixArt-Sigma-XL pretrained at 1024². **Dataset** 4.9 M LAION-HQ images centre-cropped
123 to 4096². **Hardware** $8 \times$ A100-80 GB; micro-batch 2, grad-accum 8. **ReDiF-T** Chunk 256, 36
124 reversible pairs. **Metrics** CLIP-FID (ViT-H/14) every 5 k steps on 10 k samples; 512-pair human
125 A/B test; gradient exactness on 64 held-out 256² images.

126 5.4 Ablations

127 Run A1-A3 and full system at 512² to measure individual contributions.

128 5.5 Robustness

129 Sobol sensitivity on learning rate, chunk size C , reversible depth; FGSM $\varepsilon = 1/255$ perturbations on
130 latents.

6 Results

6.1 Memory and speed

Across $256 - 1536^2$, ReDiF-T lowers activation+gradient memory by $2.8-3.4\times$ and total GPU RAM by $1.9-2.4\times$ versus checkpointing. On RTX-4090 it trains 1024^2 images at batch 8 within 23.8 GB, where checkpointing OOMs above 40 GB. Per-step overhead is 7-10%, but reinvesting freed memory into batch size yields 15-25% higher images/s.

6.2 Ablation study

At 512^2 : reversible only -36% memory; +blockwise -62% ; +KV cache -71% ; +optimiser shrink -77% versus baseline, confirming additive gains.

6.3 Image quality

ImageNet-512 (mean $\pm\sigma$ over three seeds): Baseline FID 3.18 ± 0.03 , ReDiF-T 3.15 ± 0.02 ; sFID 6.46 ± 0.05 vs 6.42 ± 0.04 ; IS 311.8 ± 2.1 vs 312.6 ± 1.9 . Paired t-tests give $p > 0.42$, indicating no significant difference.

6.4 4 K fine-tune

Baseline variants exceed 640 GB aggregate and abort. ReDiF-T peaks at 69 GB/GPU, finishes in 4.8 days and attains CLIP-FID 6.7 on COCO-Captions-4K, beating Make-A-Scene upsampling (7.1). Human judges prefer ReDiF-T in $55\% \pm 3\%$ of pairs. Gradient difference $\leq 1.1 \times 10^{-6}$ validates exactness.

6.5 Scalability

Peak memory grows linearly with patches until 512^2 then plateaus proportional to C , matching $O(C)$ theory. Doubling batch until OOM does not raise FID variance beyond 0.03.

6.6 Energy

Under equal FLOPs ReDiF-T shortens wall-clock by 12% and saves 10% energy thanks to higher throughput.

7 Conclusion

ReDiF-T unifies reversible computation, blockwise attention, low-rank caching, and online optimiser compression into the first sub-linear memory training framework for Diffusion Transformers. It halves to thirds peak memory relative to aggressive checkpointing, maintains exact gradients and image quality, and unlocks 1024^2 —and even 4096^2 —pixel-space training on affordable hardware. A one-line PyTorch interface makes adoption trivial. Future work includes temporal chunking for video diffusion, runtime-adaptive chunk sizing, and coupling with multi-step distillation for further wall-clock gains Salimans et al. [2024].

References

- Muralidhar Andoorveedu, Zhanda Zhu, Bojian Zheng, and Gennady Pekhimenko. Tempo: Accelerating transformer-based model training through memory footprint reduction. 2022.
- Yu Chen, Zhenming Liu, Bin Ren, and Xin Jin. On efficient constructions of checkpoints. *International Conference on Machine Learning, 2020*, 2020.
- Katherine Crowson, Stefan Andreas Baumann, Alex Birch, Tanishq Mathew Abraham, Daniel Z. Kaplan, and Enrico Shippole. Scalable high-resolution pixel-space image synthesis with hourglass diffusion transformers. 2024.

Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. 2020.

Marisa Kirisame, Steven Lyubomirsky, Altan Haan, Jennifer Brennan, Mike He, Jared Roesch, Tianqi Chen, and Zachary Tatlock. Dynamic tensor rematerialization. 2020.

Nikita Kitaev, Łukasz Kaiser, and Anselm Levskaya. Reformer: The efficient transformer. 2020.

Wenshuo Li, Xinghao Chen, Han Shu, Yehui Tang, and Yunhe Wang. Excp: Extreme llm checkpoint compression via weight-momentum joint shrinking. 2024.

Valerii Likhoshesterov, Krzysztof Choromanski, Jared Davis, Xingyou Song, and Adrian Weller. Sub-linear memory: How to make performers slim. 2020.

Hao Liu and Pieter Abbeel. Blockwise parallel transformers for large context models. 2023.

Ze Liu, Han Hu, Yutong Lin, Zhuliang Yao, Zhenda Xie, Yixuan Wei, Jia Ning, Yue Cao, Zheng Zhang, Li Dong, Furu Wei, and Baining Guo. Swin transformer v2: Scaling up capacity and resolution. *CVPR2022*, 2021.

Xinyin Ma, Gongfan Fang, Michael Bi Mi, and Xinchao Wang. Learning-to-cache: Accelerating diffusion transformer via layer caching. 2024.

William Peebles and Saining Xie. Scalable diffusion models with transformers. 2022.

Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. 2021.

Tim Salimans, Thomas Mensink, Jonathan Heek, and Emiel Hoogeboom. Multistep distillation of diffusion models via moment matching. 2024.

Hongjie Wang, Difan Liu, Yan Kang, Yijun Li, Zhe Lin, Niraj K. Jha, and Yuchen Liu. Attention-driven training-free efficiency enhancement of diffusion models. 2024.

Zhihang Yuan, Hanling Zhang, Pu Lu, Xuefei Ning, Linfeng Zhang, Tianchen Zhao, Shengen Yan, Guohao Dai, and Yu Wang. Ditfastattn: Attention compression for diffusion transformer models. 2024.

Zheng Zhan, Yushu Wu, Yifan Gong, Zichong Meng, Zhenglun Kong, Changdi Yang, Geng Yuan, Pu Zhao, Wei Niu, and Yanzhi Wang. Fast and memory-efficient video diffusion using streamlined inference. 2024.